Table of Contents

1 Introduction....1
2 Top-down layered static view....3
3 Detailed design of the touch_buttons module....5
3.1 TouchButtons Class....6
3.1.1 Purpose....6
3.1.2 Design & Threading....6
3.1.3 API Reference....6
3.1.3.1 BUTTONS Constants....6
3.1.3.2 Methods:....6
3.1.4 Error handling....6
3.1.5 Attributes for module test....6
3.2 Module test....7

1 Introduction

The touch buttons are located at the front panel of the Oradio3

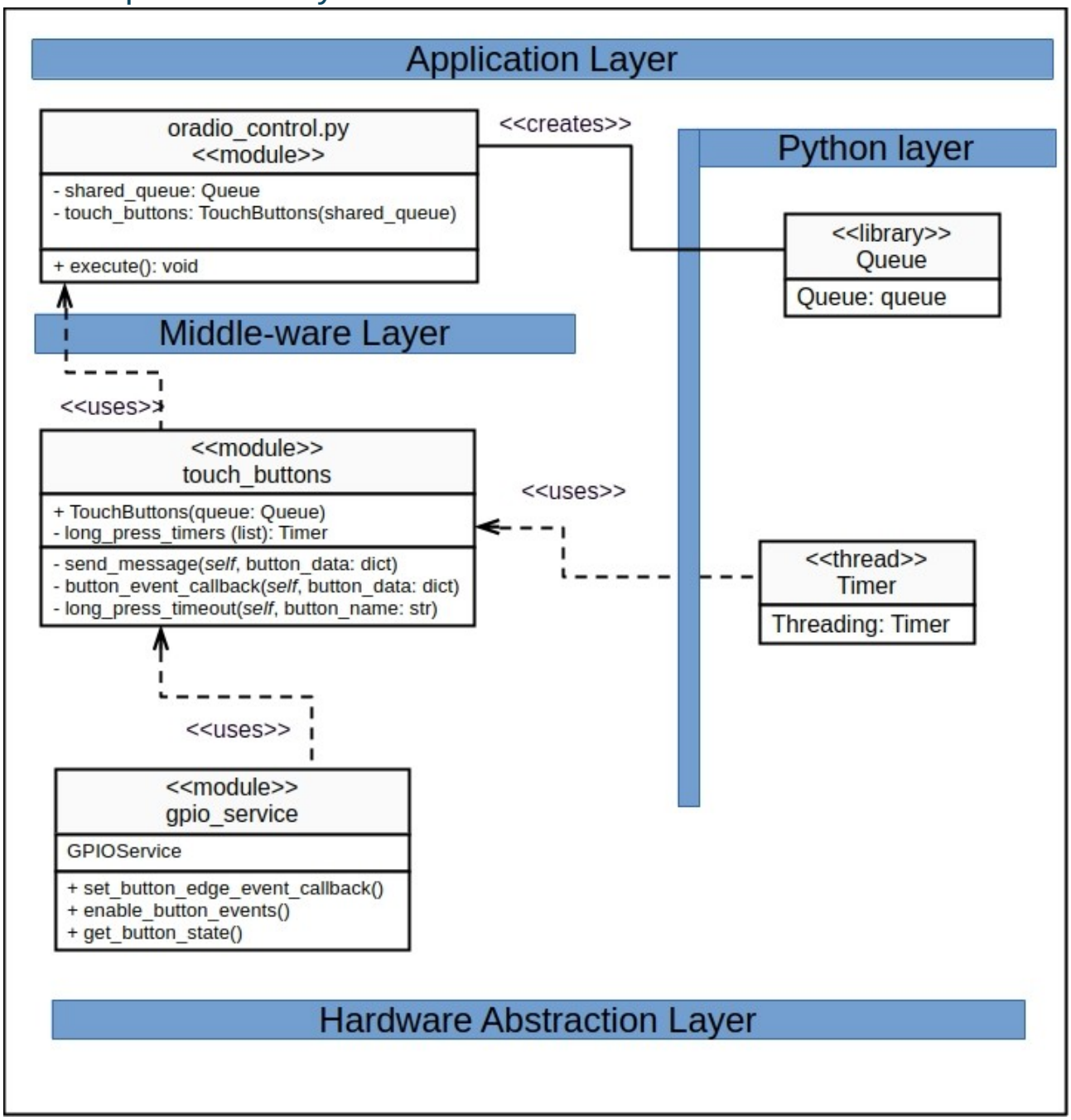
Button	Function
ButtonPlay	• Start playing the last selected music after stop • When Spotify-App is connected, the active playlist will played
ButtonStop	• The music is stops playing music. • The Oradio3 enters standby mode.
ButtonPreset1	The playlist assigned to Preset1 is played
ButtonPreset2	The playlist assigned to Preset2 is played
ButtonPreset3	The playlist assigned to Preset3 is played

The touch button has a proximity or touch sensor. A button does not need to be touched, also when in close proximity a touch will be detected.

When a button is touched a click-sound will be heard, to indicate that touching the button was detected.

When the ButtonPlay button is touched for a longer time, it will be recognized as a button-long-press. At this moment only a button-long-press is enabled for the ButtonPlay, to start the Oradio WiFi Access Point. For other button there is no button-long-press enabled.

2 Top-down layered static view



The Oradio3 software layering typically consists of three main layers:

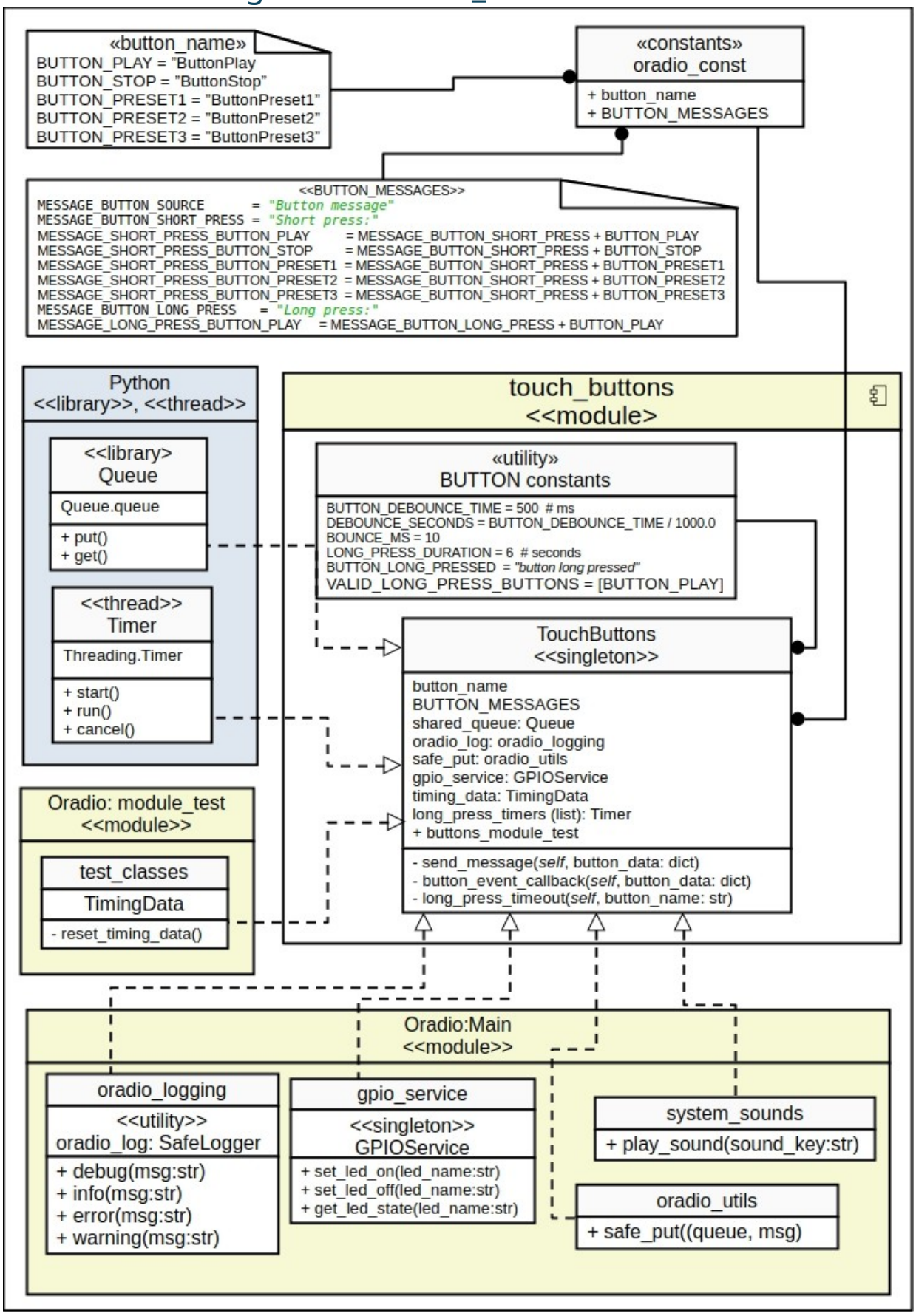
- the **hardware abstraction layer** (HAL), which includes the physical components.
- the **middle-ware layer**, which facilitates communication and data management between the hardware and applications. The OS services is considered to be part of this layer
- the **application layer**, where the actual software applications run and perform specific tasks.

The **Python Layer** is a supporting layer to include Python modules and libraries.

The TouchButtons() class is a Oradio3 module offering services related to touching buttons. It imports the GPIOService class, which provides a wrapper to setup the events when a button is touched and to read the state of the touch buttons pins.

32 The class depends on a queue, which is accepted in the constructor of the class. It is a shared queue to submit messages to the creator (oradio_control.py) of the class.

3 Detailed design of the touch_buttons module.



3.1 TouchButtons Class

3.1.1 Purpose

- Provides a method to handle the gpio_service events related to the touch buttons
- Provides a method to detect a long key press
- Provides a click sound when a button is touched
- Submits messages via the shared queue to the caller, to inform the caller on the button activities

3.1.2 Design & Threading

- **Singleton:** Restricts the class to a single instance, to offer a single point of access for the led controls.
- **Timers**: For the long press detection a Threading.Timer is used, which will expire after the long press timing.
- **Logging**: For logging purposes the oradio.SafeLogger utility is used offered by oradio_logging module.
- **Queue**: A queue is used between caller and this class to send a message upon button events.
- **GPIOService**: creates an instance of the GPIOService class to use methods to setup the event handling, and reading the state of a touch button.

3.1.3 API Reference

3.1.3.1 BUTTONS Constants

```
BUTTON_PLAY    = "ButtonPlay"
BUTTON_STOP    = "ButtonStop"
BUTTON_PRESET1 = "ButtonPreset1"
BUTTON_PRESET2 = "ButtonPreset2"
BUTTON_PRESET3 = "ButtonPreset3"
BUTTON_NAMES   = [BUTTON_PLAY, BUTTON_STOP, BUTTON_PRESET1, BUTTON_PRESET2,
                  BUTTON_PRESET3]
BUTTON_PRESSED  = "button pressed"
BUTTON_RELEASED = "button released"
BUTTON_DEBOUNCE_TIME = 500 # ms, ignore rapid repeats
DEBOUNCE_SECONDS     = BUTTON_DEBOUNCE_TIME / 1000.0
```

3.1.3.2 Methods:

There are no public methods, only private methods. The communication is managed via a shared queue, to prevent “polling” methods or callbacks.
One private method needs some more explanation, as it has a software de-bouncing feature.

Method	Arguments	Returns
`TouchButtons._button_event_callback`	`(button_data: dict))`	None
Description: • This callback is used by the GPIO event callback to inform about button events • button_data = {‘name’:str, ‘state’:str}: state = [BUTTON_PRESSED | BUTTON_RELEASED | BUTTON_LONG_PRESSED]		

name = [BUTTON_PLAY | BUTTON_STOP | BUTTON_PRESET1 | BUTTON_PRESET2 | BUTTON_PRESET3] • There is an internal software de-bouncing to prevent a new button event to come in before previous is processed. When a new button event is received within the DEBOUNCE_SECONDS it will be neglected. • When buttons_module_test is enabled the neglected event will be reported and printed on the console.
Errors:

3.1.4 Error handling

- All errors are logged using the oradio_log.error or .warning utility of oradio_logging.py

3.1.5 Attributes for module test

The class provides an attribute <**buttons_module_test** > to enable or disable module test functions within the class. This is used when class is created to add a TimingData() class (test_classes.py) for performance measurements.

3.2 Module test

There is a module test program <touch_buttons_test.py> that will provide following tests to test the methods of the class:

- Press a button and check if a message is in the queue
- For each button the GPIO button callback is triggered, and the message queue is checked
- Simulate button press and release using GPIO, to test timings
- Buttons events, with GPIO callback and performance measurement, including neglected callbacks.